Introduction to Artificial intelligence

Tutorial Sheet 5 (Adversarial Search and CSPs - solution)

Exercise 1:

This problem exercises the basic concepts of game playing, using tic-tac-toe (noughts and crosses) as an example. We define X_n as the number of rows, columns, or diagonals with exactly n X's and no O's at all. Similarly, O_n is the number of rows, columns, or diagonals with just n O's. The utility function assigns +1 to any position with $X_3 = 1$ and -1 to any position with $O_3 = 1$. All other terminal positions have utility 0. For non-terminal positions, we use a linear evaluation function defined as $Eval(s) = 3X_2(s) + X_1(s) - (3O_2(s) + O_1(s))$.

a. Approximately how many possible games of tic-tac-toe are there? Do a detailed, precise analysis.
b. Show the whole game tree starting from an empty board down to depth 2 (i.e., one X and one O on the board), taking symmetry into account.
c. Mark on your tree the evaluations of all the positions at depth 2.
d. Using the minimax algorithm, mark on your tree the backed-up values for the positions at depths 1 and 0, and use those values to choose the best starting move.
e. Circle the nodes at depth 2 that would not be evaluated if alpha–beta pruning were applied, assuming the nodes are generated in the optimal order for alpha–beta pruning.

---

a. Since, there are 9 possible ways of placing the first mark on the square, 8 remaining ways of placing the second, 7 remaining ways of placing the third, so on….and 1 remaining for ninth one.
Therefore, this would be 9*8*7*6*5*4*3*2*1=9! ways, which is equal to 362880.

But this number is clearly too high, because there are other possibilities like the game can be finished in 5 moves, which is the least number of moves.
So, to find the all possible games, it is enough to find the number of games end in 5 moves, number of games end in 6 moves, number of games end in 7 moves, number of games end in 8 moves and number of games end or draw in 9 moves.

Number of games ending on the 5th move:

• Assume 'X' starts the game. Then after the 5th move, there will be three X's and two O's on the board. The game ends, if the three X's are placed vertically or horizontally or diagonally.
• There are 8 possibilities of placing three squares (3 vertical, 3 horizontals and 2 diagonals), and it does not matter in which order the three 'X' are placed (so 3!) and the 2 'O' could go into the remaining six squares in any order, (so 6*5). Thus, the number of possibilities for the game completed after 5 moves is 1440 (8*3! *6*5).

Number of games ending on the 6 th move:

• After the 6th move there will be three X's and three O's on the board. On the sixth move, O is placed. The game ends if three O's are placed vertically or horizontally or diagonally.
• There are 8 possibilities of placing three squares (3 verticals, 3 horizontals, 2 diagonals) and it does not matter in which order the three 'O's are placed and the 3 'X's could go into the remaining six squares in any order. So the number of games ending on the sixth move can be calculated as follows: 8*3!*6*5*4 = 5760
• But, these 5760 include the cases where three O's are in a row and three X's are in a row (if three X's are in a row, this means that the game should have already ended on the 5th move). So, we must exclude such cases. If three O's are placed diagonally, there is no possibility of three X's in a row. So, these cases occur only when three O's are placed horizontally or vertically.
• There are 3 horizontal and 3 vertical rows to first place three O's (total six ways) and the order is not important (so, 6*3!). After placing three O's, there are two empty rows and two empty columns to place three X's and the order does no matter (so, 2*3!). Thus, the number of cases where 3 X's and 3 O's placed in a row= 6*3!*2*3!
Thus, the number of games that end in six moves is: 5760 – 432 = 5328.
step: 2 of 7

Number of games that end on the 7th move:

• After the 7th move, there will be four X's and three O's on the board. On the seventh move X is placed on the board. The game ends if the 4th X is placed such that it is in a line of X's (vertically or horizontally or diagonally).

• Since, the 4th X must be in line with other 2 X's, the order of placing four X's is a matter in this case. Thus placing four X's such that the fourth can be in line with other 2 X's can be done in 8*3*6*3!.
• Three O's can be placed in the remaining five empty squares (5*4*3). Then the number of games ending upon the 7th move is 8*3*6*3!*5*4*3= 51840. But it is required to exclude the number of cases where three X's and three O's are in a row, which is 6*3*6*3!*3! = 3888.
Thus, the number of games ending in a win on seventh move = 51840-3888=47952.

Number of games ends on the 8th move:

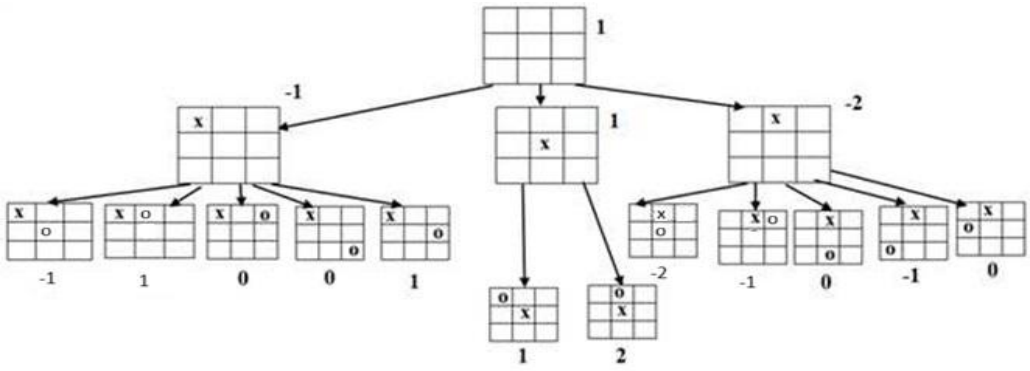
• After 8th move there will be four X's and four O's on the board. In the eighth move O is placed on the board. The game ends if 4th O's placed such that it is in a line of O's(vertically or horizontally or diagonally).
• Similar to previous step, the number of games that end in the 8th move= 8*3*6*3!*5*4*3*2 = 103680. But it is required to exclude the number of cases where three X's and three O's are in a row, which is 6*3*6*3!*2*4! = 31104.
Thus, the number of games ending in a win on 8th move = 103680-31104= 72576.

Number of games ends on the 9th move:

• After 9th move there will be five X's and four O's on the board. In the 9th move X is placed on the board. The game ends if 5th X's is placed such that it is in a line of X's(vertically or horizontally or diagonally).
• On the 9th move, either the game end in draw or win.
• The number of games end in win on placing X and it makes a line of three squares diagonally= 2*3*8*4!*4! = 27648.
• The number of games end in win on placing X and it makes a line of three squares either vertically or horizontally= 6*3*4*4!*4! = 41472.
• The number of games end in win where placing X makes two distinct three X's in a row, where those two intersect = 22*1*4!*4! = 12672.
• The number of games end in draw on the 9th move=16*5!*4! = 46080.
Thus, the number of games ending in a win or draw on the 9th move =27648+41472+12672+46080= 81792+46080=127872.

Therefore, the possible games of tic-tac-toe are, 1440 + 5328 + 47952 + 72576 + 127872= 255168.

b- The game tree starts with an empty board (depth 0). Generally, in the next level (depth 1) there would be 9 states, as X can be placed in any one of nine empty boxes. But, by considering symmetry, there will be only three states, as the remaining six states are symmetrical. Without symmetry, in the 2nd level (depth 2), there should be 9x8=72 states. But, by considering symmetry, there will be only 12 states, as the remaining sixty states are symmetrical.



c- Now, calculate the evaluation for leftmost state at depth 2. Eval(s) = 3X2(s) + X1(s) - (3O2(s) + O1(s)) = 0 + 2 - (0+1) = 1 and the evaluation for the next node to leftmost node, at depth 2. Eval(s) = $3X_2(s) + X_1(s) - (3O_2(s) + O_1(s))$ = 0 + 2 - (0 + 3) = 2 – 3 = -1

d- The nodes at depth 1 are MIN nodes. Thus the backed up values of each node will be the minimum of values of the successor nodes.
For the leftmost node, the backed up value or MINI-MAX-VALUE= min{1,-1,0,0,1} = -1
For the middle node, the backed up value or MINI-MAX-VALUE = min{1,2} = 1

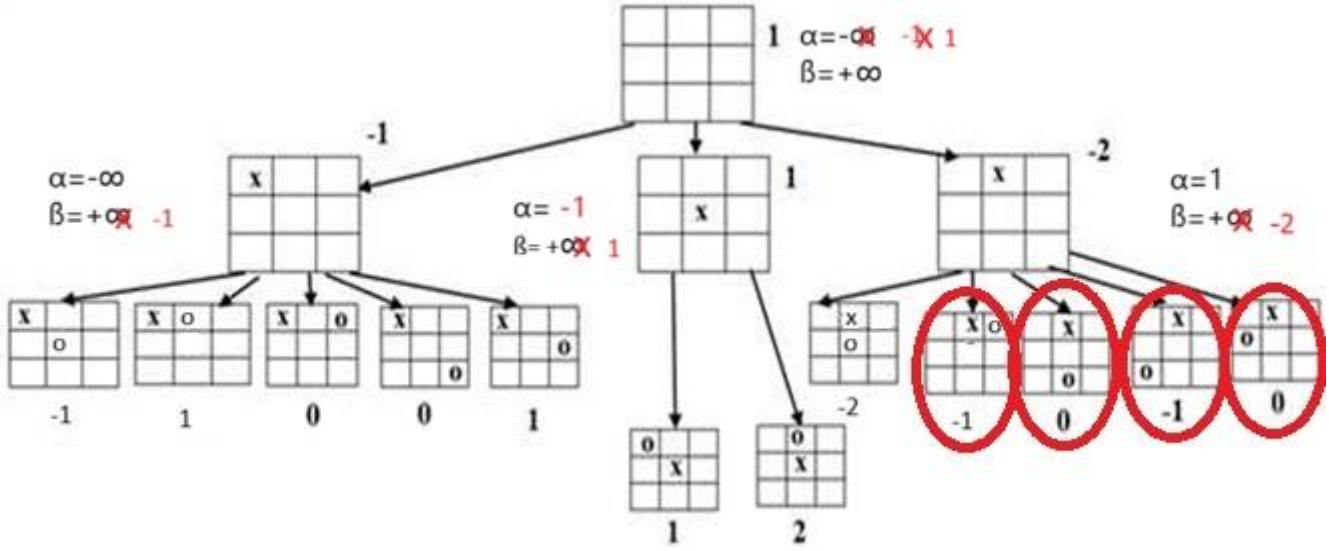
For the right most node, the backed up value or MINI-MAX-VALUE = min{-1,-2,0,-1,0} = -2
• The root node at depth 0 is MAX node. Thus the backed up value of the root node will be the maximum of values of the successor nodes.
The backed up value of the root node or MINI-MAX-VALUE =max{-1,1,-2}=1

• Since the backed up value of the root is from the successor node that has X at the center, the best move is to place X at the center position first.
Thus the middle node (placing X in the center) is the best move.

e- Alpha-beta pruning works similar to the minimax algorithm with the exception that at each step Alpha-beta prunes the branches that do not affect the final decision.

MINI-MAX-VALUE(root)= max{ min{1,-1,0,0,1}, min{1,2}, min{-1,-2,0,-1,0}}=max{-1,min{1,2},-2}=1

The MIN-MAX value of the root depends on the four nodes that have values -1,1,2 and -2. Thus, the Alpha-beta pruning algorithm prunes the remaining eight nodes that do not effect the final value.

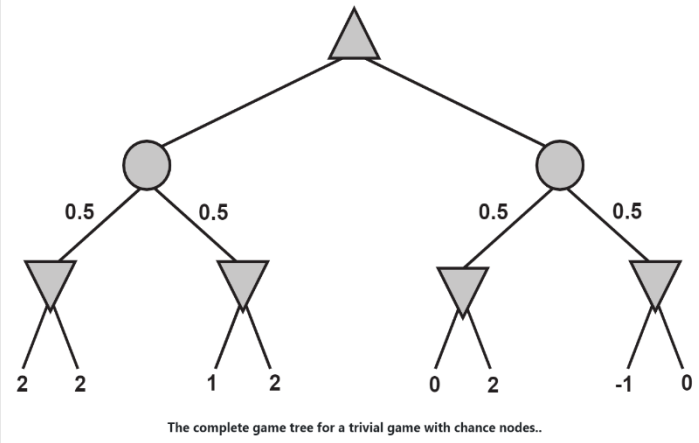


The nodes circled indicate the nodes that are to be pruned. Thus, these circled nodes nor not evaluated by Alpha-beta pruning algorithm.

Exercise 2:
This question considers pruning in games with chance nodes. The figure below shows the complete game tree for a trivial game. Assume that the leaf nodes are to be evaluated in a left-to right order, and that before a leaf node is evaluated, we know nothing about its value—the range of possible values is -∞ to +∞.
a. Copy the figure, mark the value of all the internal nodes, and indicate the best move at the root with an arrow.
b. Given the values of the first six leaves, do we need to evaluate the seventh and eighth leaves? Given the values of the first seven leaves, do we need to evaluate the eighth leaf? Explain your answers.
c. Suppose the leaf node values are known to lie between –2 and 2 inclusive. After the first two leaves are evaluated, what is the value range for the left-hand chance node?
d. Circle all the leaves that need not be evaluated under the assumption in (c).

_____
_____

a-



The complete game tree for a trivial game with chance nodes..

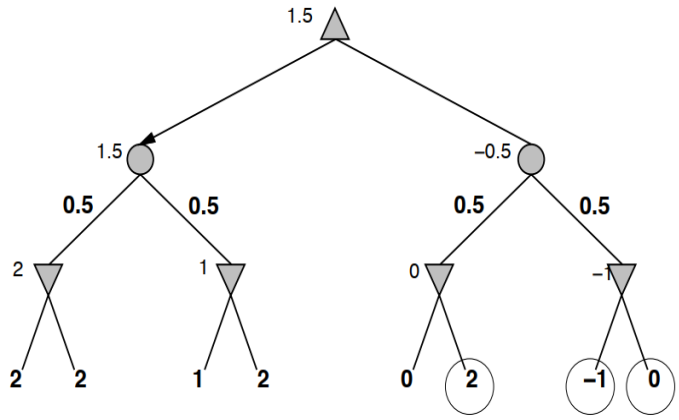


Figure S5.5 Pruning with chance nodes solution.

b. Given nodes 1–6, we would need to look at 7 and 8: if they were both $+\infty$ then the values of the min node and chance node above would also be $+\infty$ and the best move would change. Given nodes 1–7, we do not need to look at 8. Even if it is $+\infty$, the min node cannot be worth more than -1, so the chance node above cannot be worth more than -0.5, so the best move won't change.

c. The worst case is if either of the third and fourth leaves is -2, in which case the chance node above is 0. The best case is where they are both 2, then the chance node has value 2. So it must lie between 0 and 2.

d. See figure.

Exercise 3:

Mom, Dad, Baby, Student, Teacher, and Guide are lining up next to each other in six linear spots labeled 1 to 6, one to a spot. Baby needs to line up between Mom and Dad. Student and Teacher need to be next to each other. Guide needs to be at one end, in spot 1 or 6.

- Formulate this problem as a CSP: list the variables, their domains, and the constraints. Encode unary constraints as a constraint rather than pruning the domain. (No need to solve the problem, just provide variables, domains and constraints.)

Variables:

- G: The position assigned to the person "Guide"
- M: The position assigned to the person "Mom"
- D: The position assigned to the person "Dad"
- B: The position assigned to the person "Baby"
- S: The position assigned to the person "Student"
- T: The position assigned to the person "Teacher"

Domains:

- DG = DM = DD = DB = DS = DT = {1, 2, 3, 4, 5, 6}

Constraints:

1. All variables must have different values (alldifferent constraint): G ≠ M ≠ D ≠ B ≠ S ≠ T
2. Baby must be between Mom and Dad (binary constraint): ((M < B < D) ∨ (M > B > D)) ➔

   (M=B+1 and D=B-1) Or (M=B-1 and D=B+1)
3. Student and Teacher must be next to each other (binary constraint): abs(S - T) = 1
4. Guide must be at one end (unary constraint): (G = 1) ∨ (G = 6)
5. Unary constraints (to avoid pruning the domains): M ∈ {1, 2, 3, 4, 5, 6} D ∈ {1, 2, 3, 4, 5, 6} B ∈ {1,2, 3, 4, 5,6 } S ∈ {1, 2, 3, 4, 5, 6} T ∈ {1, 2, 3, 4, 5, 6}

Exercise 4:

Consider the problem of placing k knights on an n × n chessboard such that no two knights are attacking each other, where k is given and $k \leq n^2$.

a. Choose a CSP formulation. In your formulation, what are the variables?

b. What are the possible values of each variable?

c. What sets of variables are constrained, and how?

d. Now consider the problem of putting as many knights as possible on the board without any attacks. Explain how to solve this with local search by defining appropriate ACTIONS and RESULT functions and a sensible objective function.

---

a. In the problem of placing k knights on an n × n chessboard such that no two knights are attacking each other, a suitable CSP formulation would involve representing each knight as a variable.

The variables in this formulation would be $X_1, X_2, X_3, ..., X_k$, where k is the number of knights to be placed on the board.

b. The possible values for each variable Xi (representing the position of the i−th knight) would be the set of all valid square positions on the n × n chessboard. This set can be represented as {(r, c) | 1 ≤ r ≤ n, 1 ≤ c ≤ n}, where r and c represent the row and column indices, respectively.

c. The constraints in this problem arise from the condition that no two knights should be attacking each other. The sets of variables that are constrained are determined by the legal move patterns of a knight on a chessboard.

A knight can move to any square that is two squares away horizontally and one square vertically, or two squares vertically and one square horizontally. Therefore, for any two variables X_i and X_j, representing the positions of the ith and j^{th} knights, respectively, the constraint would be:

NOT ((abs(X_i.row - X_j.row) == 2 AND abs(X_i.col - X_j.col) == 1) OR (abs(X_i.row - X_j.row) == 1 AND abs(X_i.col - X_j.col) == 2))

This constraint ensures that the positions represented by Xi and Xj are not in an attacking position with respect to each other.

d. To solve the problem of putting as many knights as possible on the board without any attacks using local search, we can define the following components:

1. State representation: A state in the search space would be a configuration of knights on the board, represented by a set of board positions occupied by the knights.
2. ACTIONS: The possible actions in a given state would be to add a new knight to an empty square on the board, or to move an existing knight to a different empty square.
3. RESULT function: The RESULT function takes a state and an action as input and produces a new state by applying the action to the current state. For example, if the action is to add a new knight at position (r, c), the RESULT function would add (r, c) to the set of occupied positions. If the action is to move a knight from position (r1, c1) to (r2, c2), the RESULT function would remove (r1, c1) from the set and add (r2, c2).
4. Objective function: The objective function should evaluate the quality of a given state, with the goal of maximizing the number of knights on the board while satisfying the constraint of no attacks. One possible objective function could be: f(state) = (number of knights on the board) - (number of pairs of attacking knights) The goal would be to maximize this objective function.
5. Local search algorithm: With the above components defined, a suitable local search algorithm, such as hill-climbing or simulated annealing, can be employed to find a (possibly suboptimal) solution that maximizes the number of knights on the board without any attacks.

The local search algorithm would start with an initial state (e.g., an empty board or a random configuration of knights) and iteratively apply the ACTIONS to generate new states. The objective function would guide the search towards states with a higher number of knights and fewer attacking pairs.